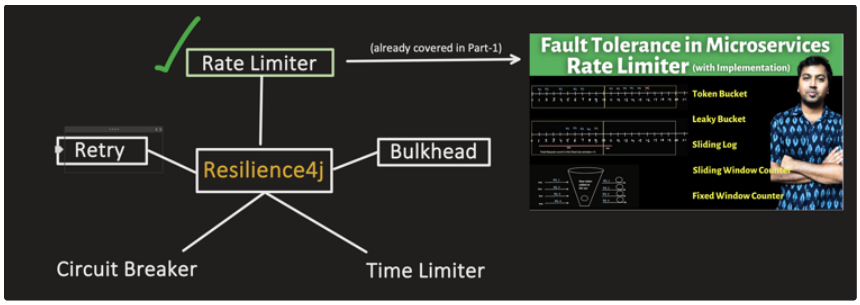
# Bulkhead: Fault-Tolerant Microservice (Part-2)

To build Fault tolerant microservices: Resilience4j provides below mechanisms




Lets see in depth of:

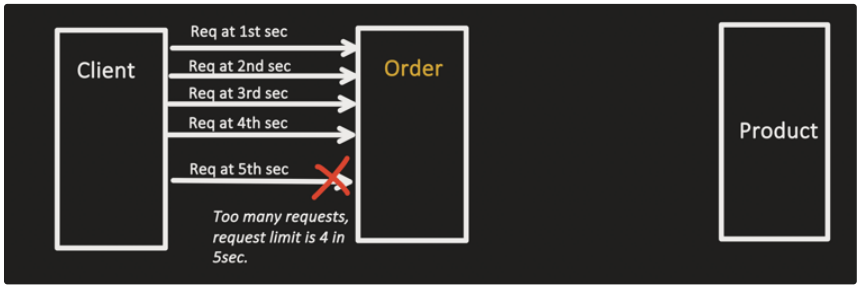
Bulkhead**:**

**Rate Limiter:**

- Protects **our application from our clients** by limiting how many requests we accept within a specific time window from them.
- also Rate limiter never talks about the concurrency.

Concept && Coding

*Ex: let say our application is "Order".*

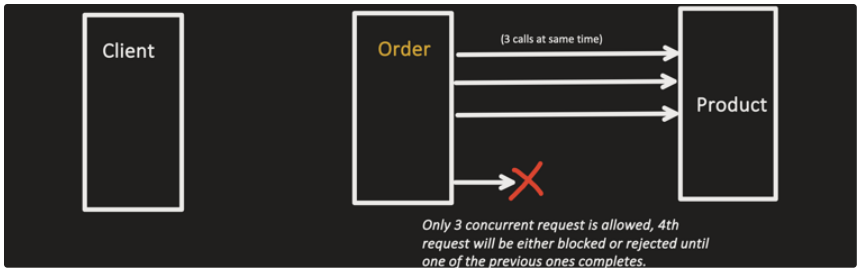



**Bulkhead:**

- It helps to control **how many concurrent requests can go to downstream service**.

*Ex: let say our application is "Order"*

*Use-Case: Let say, "Product" is small and light weight service and can only handle 3 concurrent request.*




Above use case is resolved through "Semaphore Bulkhead"

It also protects **our application from our downstream services** by limiting how many threads we allocate to them.

*<u>Use-Case:</u> Let say our Order Service has 2 APIs (endpoints)*

***API 1*** *: make a downstream call to Product service.*

***API 2*** *: make a downstream call to Payment service.*

*Product service takes ~100 milliseconds to process a request (Fast Service)*

*and*

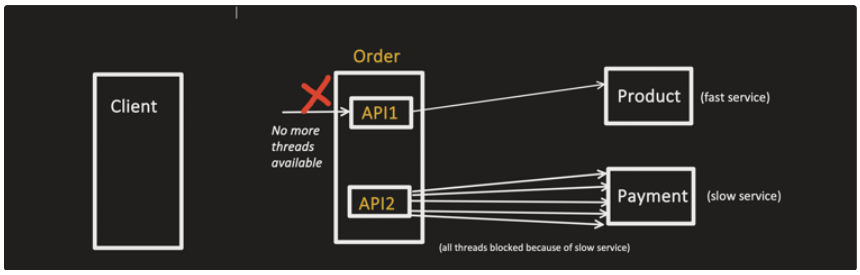
*Payment service takes ~5second to process a request (Slow service)*

*What if there is a sudden spike in **API 2** traffic and more and more calls are made to Payment service (say calls are not concurrent).*

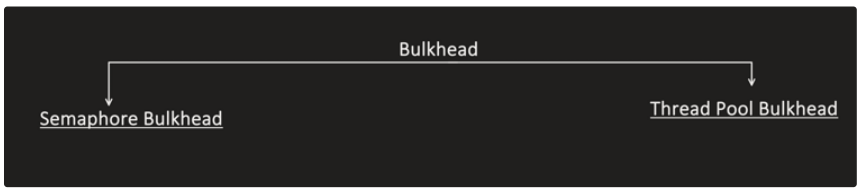
Concept && Coding

*Since Payment service is slow, it holds or blocks the thread longer( for ~5seconds) and its possible that all the threads of Order service now blocked (by Payment service only).*

*Now even the Order service fast endpoint **(API 1)** becomes unavailable as there is no threads available.*



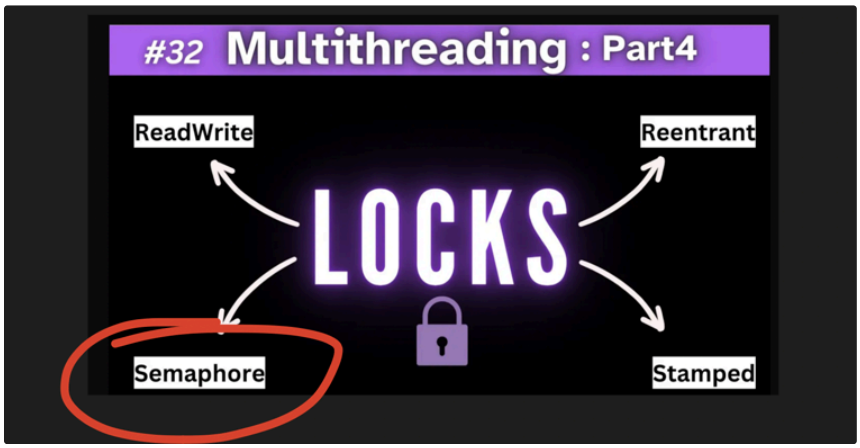

Above use case is resolved through "Thread Pool Bulkhead"




**<u>Semaphore Bulkhead:</u>**

- Limits the number of concurrent calls using counter.
- If the limit is reached, further calls are rejected immediately or blocked for specific wait time.

It limits the number of concurrency call, via Semaphore Lock.
I have already explained in depth about Semaphores in Multithreading topic of Java playlist. Have a look if there is any doubt with Semaphore locks.




Pom.xml dependency

```
<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-boot3</artifactId>
    <version>2.1.0</version>
</dependency>

```

Concept && Coding

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    OrderService orderService;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {
        orderService.invokeProductAPI(id);
    }
}

```

```
@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping(value = "/products/{id}")
    String getProductById(@PathVariable("id") String id);
}

```

```
@Component
public class OrderService {

    @Autowired
    ProductClient productClient;

    @Bulkhead(
        name = "productService",
```

```java
        type = Bulkhead.Type.SEMAPHORE,
        fallbackMethod = "productFallback"
    )
    public void invokeProductAPI(String id) {
        String response = productClient.getProductById(id);
        System.out.println("Response from Product api call is: " + response);
    }

    public void productFallback(String id, Throwable t) {
        System.out.println("too many concurrent request, pls try again later");
        // throw exception here and handle it gracefully
    }
}

```

application.properties

```properties
server.port=8081

spring.application.name=order-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

#only 2 concurrent calls allowed and
#for further req, if there is no thread available, request will fail immediately
resilience4j.bulkhead.instances.productService.maxConcurrentCalls=2
resilience4j.bulkhead.instances.productService.maxWaitDuration=0

```

Concept && Coding

**Accepted values for maxWaitDuration:** It accepts values in the following format:

<number><time-unit>

**Supported time units:**

- **ms** — milliseconds
- **s** — seconds
- **m** — minutes
- **h** — hours
- **d** — days (not typical, but valid)

Example:

resilience4j.bulkhead.instances.productService.maxWaitDuration=0 # Reject immediately

resilience4j.bulkhead.instances.productService.maxWaitDuration=300ms # Wait 300 milliseconds

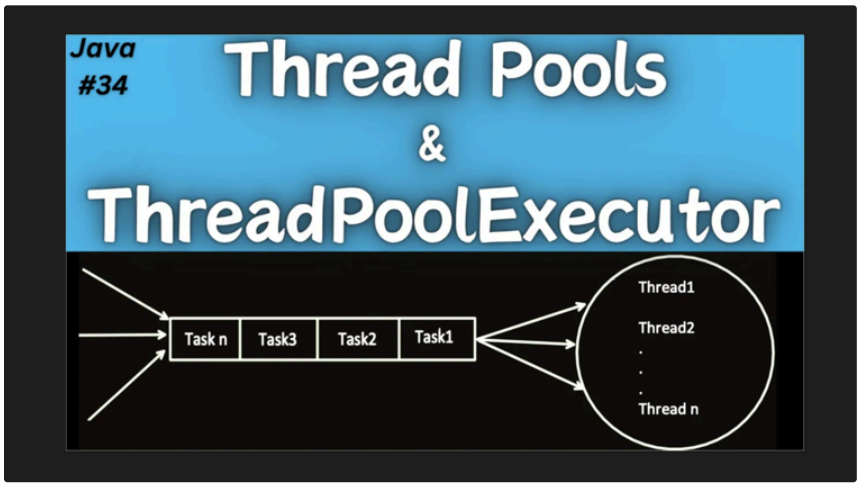
resilience4j.bulkhead.instances.productService.maxWaitDuration=2s # Wait 2 seconds

resilience4j.bulkhead.instances.productService.maxWaitDuration=1m # Wait 1 minute

**<u>Thread Pool Bulkhead:</u>**

- Assign a dedicated thread pool for each service.
- Only that pool is used to call the downstream.

If you have any doubts, how Thread Pools and Thread Pool Executor works, kindly have a look at this video in Java playlist, I have already discussed in depth.




```java
@Component
public class OrderService {

    @Autowired
    ProductClient productClient;
  /*
    • The AOP proxy intercepts your method call.
    • It submits your method to the Bulkhead's thread pool
    (configured via application.properties).

    CompletableFuture.supplyAsync(() -> {
    return ourMethodLogic();  // The whole method body runs here
    }, bulkheadThreadPoolExecutor);

    • It returns a CompletableFuture to the caller for async response.
    • Inside your method, CompletableFuture.completedFuture(...)
      just wraps the result in a completed future —
      it does NOT run anything in a thread pool.
    */
    @Bulkhead(
        name = "productService",
        type = Bulkhead.Type.THREADPOOL,
        fallbackMethod = "productFallback"
    )
    public CompletableFuture<String> invokeProductAPI(String id) {
        return CompletableFuture.completedFuture(
                productClient.getProductById(id)
        );
    }

    public CompletableFuture<String> productFallback(String id, Throwable t) {
        // even though its async, we need to properly handle the exception
        // else our client will get 500 response with BulkheadFullException
        return CompletableFuture.completedFuture("Product Service is busy");
    }
}

```

Concept && Coding

application.properties

```properties
server.port=8081
```

```

spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

# max 3 threads in the pool and that's the max pool size too.
# if all 3 threads are busy, max 2 more requests will be put into queue
# from 6th request onwards, it will start rejecting the requests.
resilience4j.thread-pool-bulkhead.instances.productService.coreThreadPoolSize=3
resilience4j.thread-pool-bulkhead.instances.productService.maxThreadPoolSize=3
resilience4j.thread-pool-bulkhead.instances.productService.queueCapacity=2

```

ThreadPool for productService will be created with these configurations.

Testing the Output:

```java
@Component
public class OrderService {

    @Autowired
    ProductClient productClient;

    @Bulkhead(
        name = "productService",
        type = Bulkhead.Type.THREADPOOL,
        fallbackMethod = "productFallback"
    )
    public CompletableFuture<String> invokeProductAPI(String id) {
        System.out.println("Thread name is:" + Thread.currentThread().getName());
        return CompletableFuture.completedFuture(
                productClient.getProductById(id)
        );
    }

    public CompletableFuture<String> productFallback(String id, Throwable t) {
        // even though its async, we need to properly handle the exception
        // else our client will get 500 response with BulkheadFullException
        System.out.println("Product Service is busy");
        return CompletableFuture.completedFuture("Product Service is busy");
    }
}

```

Concept && Coding

Output:

*Max thread pool size is = 3 and Queue size is = 2*

*Invoked the API 6 times, lets see the output:*

```
2025-07-17T10:15:08.998+05:30  INFO 82435 --- [order-service] [           main] c.c.o.OrderserviceApplication           : Started OrderserviceApplication in 1.48
2025-07-17T10:20:08.830+05:30  INFO 82435 --- [order-service] [trap-executor-0] c.n.d.s.r.aws.ConfigClusterResolver     : Resolving eureka endpoints via configur
2025-07-17T10:20:09.593+05:30  INFO 82435 --- [order-service] [nio-8081-exec-2] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet '
2025-07-17T10:20:09.593+05:30  INFO 82435 --- [order-service] [nio-8081-exec-2] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet
2025-07-17T10:20:09.594+05:30  INFO 82435 --- [order-service] [nio-8081-exec-2] o.s.web.servlet.DispatcherServlet        : Completed initialization in 1 ms
Thread name is:bulkhead-productService-1 —— Thread1, RQ1
Thread name is:bulkhead-productService-2 —— Thread2, RQ2
Thread name is:bulkhead-productService-3 —— Thread3, RQ3   and RQ4 and RQ5 is inserted into the queue
Product Service is busy   RQ6 is rejected, as there is no thread available and no space in queue
Thread name is:bulkhead-productService-1 —— Thread1 becomes free and picked RQ4 from queue
Thread name is:bulkhead-productService-2 —— Thread2 becomes free and picked RQ5 from queue
```

**Time Limiter:**

- Time Limiter is used to prevent async call from hanging indefinitely.
- Time Limiter is non blocking in Resilience4j.
- Means, it is mainly designed for asynchronous operations, that returns a reactive type like Mono, Flux etc.

But currently we have covered Blocking calls like RestTemplate, RestClient and FeignClient.

And we have already covered how to control the time out of the blocking calls.

Feign Client example:

application.properties

```
#request and connection timeout applicable to only Product-service FeignClient
feign.client.config.product-service.connectTimeout=3000
feign.client.config.product-service.readTimeout=5000
```

Time Limiter context is same but for reactive calls. Will cover TimeLimiter, when will start Spring Web flux (reactive programming)

Concept && Coding